Initiation au Développement : Algorithmique Fondamentale, Mémoire RAM et Structures de Contrôle

Thème principal : Variables, Quadruplet Mémoire, Affectation Destructive, Conditionnelles et Boucles (Pour / TantQue)

Synthèse exécutive : Ce cours pose les fondations absolues de la pensée algorithmique et de la programmation en C++ pour le BUT Informatique. Il résout le problème fondamental de la traduction d'un processus logique ou décisionnel concret en une séquence d'instructions formelles, explicites et déterministes exécutables par un ordinateur. La séance s'articule autour de trois piliers principaux : la modélisation de la mémoire vive (RAM) à travers le quadruplet d'une variable (Nom, Type, Valeur, Adresse) et l'affectation destructive ; la prise de décision conditionnelle guidée par la logique booléenne (Si / Alors / Sinon) ; et l'automatisation des traitements répétitifs au moyen de boucles bornées (Pour) et non bornées (TantQue).

1. Mémorisation et Manipulation des Données : Les Variables et la Mémoire RAM

En informatique, une variable ne doit pas être confondue avec une variable mathématique statique. Il s'agit d'un emplacement mémoire physique étiqueté dans la mémoire vive (RAM) de la machine. Toute variable est strictement caractérisée par un quadruplet : son Nom (qui permet au programmeur de s'y référer), son Type (qui détermine l'encodage binaire et la quantité d'octets allouée), sa Valeur (le résultat binaire à un instant t) et son Adresse mémoire physique. Lors de l'exécution d'une instruction d'affectation (par exemple 'x ← x + 1'), le processeur suit un protocole rigoureux en deux étapes chronologiques : 1. Évaluation : Il lit en RAM les valeurs actuelles des variables de l'expression située à droite et effectue le calcul. 2. Écriture : Il inscrit ce résultat final dans la case mémoire désignée à gauche. Ce processus est strictement destructif : la nouvelle valeur écrase irrémédiablement l'ancienne. En partiel, le piège le plus critique concerne la lecture d'une variable non initialisée. Sans affectation initiale, le processeur lit les octets résiduels laissés par d'autres applications en RAM (la 'valeur poubelle'), ce qui fausse intégralement les calculs.

- Quadruplet indissociable d'une variable : Nom (identificateur), Type (format et taille en octets), Valeur (contenu binaire) et Adresse physique (emplacement hexadécimal en RAM).
- Affectation destructive (← ou =) : Évaluation chronologique du membre de droite puis écriture dans la case mémoire du membre de gauche.
- Valeur poubelle : Bits résiduels non nettoyés présents en RAM, lus si une variable est exploitée sans initialisation préalable.

Cas concret enseignant : *La métaphore de l'armoire d'archivage à tiroirs : Imaginons la mémoire RAM comme une grande armoire d'archivage contenant des tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner sa profondeur selon le type choisi ('Entier'). L'instruction d'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir précis, à jeter au recyclage le morceau de papier qui s'y trouvait (l'ancienne valeur), et à y déposer une nouvelle feuille portant l'inscription 150.*

2. Prise de Décision : Structures Conditionnelles et Logique Booléenne

Par défaut, l'ordinateur exécute les instructions d'un programme de façon purement séquentielle, ligne par ligne. Pour introduire des comportements adaptatifs, on fait appel aux structures conditionnelles. Lorsqu'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]' est rencontré, l'unité centrale commence par évaluer le prédicat booléen : - Si la condition donne VRAI, le processeur exécute uniquement le [Bloc A], puis reprend le cours normal du programme en ignorant le [Bloc B]. - Si la condition donne FAUX, le [Bloc A] est ignoré et seul le [Bloc B] (s'il existe) est exécuté. Pour composer des conditions complexes, la maîtrise des opérateurs booléens est impérative : l'opérateur ET (AND) exige que toutes les sub-conditions soient simultanément vraies pour valider le test ; l'opérateur OU (OR) nécessite qu'au moins une condition soit vraie ; l'opérateur NON (NOT) inverse la valeur de

vérité.

• Structure d'alternative (Si / Alors / Sinon) : Aiguillage logique orientant le flux d'exécution séquentiel selon le résultat d'un test.

• Expression Booléenne (Prédicat) : Proposition logique évaluée uniquement à VRAI (True) ou FAUX (False).

• Opérateurs Logiques (ET, OU, NON) : Connecteurs régis par leurs tables de vérité pour assembler des conditions complexes.

Cas concret enseignant : *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'expulser les billets de banque, l'automate évalue la condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles_dans_automate >= montant_demandé). Si cette expression globale s'évalue à VRAI, le paiement est validé et les billets sont distribués ; SINON, l'opération est refusée, un message d'erreur est affiché sur l'écran et la carte bancaire est immédiatement restituée à l'usager.*

3. Répétition et Automatisation : Structures Itératives (Pour et TantQue)

La puissance de calcul informatique repose sur la capacité du processeur à répéter des séries d'instructions à très haute vitesse et sans erreur. On distingue deux grandes familles de structures itératives : 1. La boucle POUR : Elle est privilégiée quand le nombre d'itérations est déterminé avant d'entrer dans la boucle. Elle intègre de façon native l'initialisation du compteur, le test de limite et l'incrémentation automatique à chaque passage. 2. La boucle TANT QUE : Elle est employée lorsque le nombre de répétitions dépend d'un événement dynamique au cours du programme (ex: valider la saisie d'un mot de passe). La condition est évaluée AVANT chaque passage. Si elle est vraie, le corps s'exécute ; si elle est fausse au départ, le corps n'est jamais exécuté. La règle d'or d'une boucle TANT QUE est d'assurer la modification, à l'intérieur du corps de la boucle, d'au moins une variable engagée dans la condition de sortie. Dans le cas contraire, la condition reste éternellement VRAIE, provoquant une 'boucle infinie' qui gèle l'application et consomme 100% des ressources CPU.

• Boucle POUR (FOR) : Structure itérative bornée et déterministe, utilisée lorsque le nombre précis de répétitions est connu à l'avance.

• Boucle TANT QUE (WHILE) : Structure itérative non bornée et dynamique, répétée tant qu'une condition logique demeure VRAIE.

• Incrémentation & Boucle Infinie : Obligation de modifier la variable de contrôle dans un TantQue sous peine de geler l'application.

Cas concret enseignant : *La métaphore des consignes de l'entraîneur d'athlétisme : Si l'entraîneur ordonne au coureur 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est fixé au départ et la sortie est déterministe). S'il ordonne 'Cours tant qu'il ne pleut pas', il s'agit d'une boucle TANT QUE (la fin de l'exercice dépend d'une observation météo réévaluée à chaque tour).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' (ou 'x ← x + 1') est-elle valide en informatique alors qu'elle est mathématiquement impossible ?
Réponse du professeur : En programmation, le signe '=' (ou '←') ne représente pas une égalité algébrique statique, mais une opération d'affectation destructive en deux étapes chronologiques. Le processeur évalue d'abord la partie droite en lisant la valeur actuelle de x en RAM (ex: 5) et y ajoute 1 (obtenant 6). Ensuite, il écrit ce résultat 6 dans la case mémoire x à gauche, écrasant l'ancienne valeur 5.

Question étudiant : Que se passe-t-il concrètement en mémoire RAM si j'oublie de modifier la variable de contrôle dans le corps d'une boucle TantQue ?
Réponse du professeur : Si la variable de contrôle n'est pas modifiée dans le corps de la boucle, la condition d'arrêt reste indéfiniment VRAIE. Le processeur va réexécuter le bloc d'instructions indéfiniment sans jamais pouvoir sortir. C'est le phénomène de boucle infinie, qui bloque l'exécution du programme et nécessite l'arrêt forcé du processus.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme C++ complet (selon la norme formelle de l'IUT d'Anglet : cartouche d'en-tête, tripartition, flèches de flot de données et style Allman) qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

Démarche de résolution :
Étape 1 : Rédaction du cartouche d'en-tête obligatoire (nom du fichier, auteur, date, objectif). Étape 2 : Découpage de la fonction main() selon la tripartition réglementaire IUT (// CONSTANTES, // VARIABLES, // TRAITEMENTS). Étape 3 : Déclaration explicite des variables n, i et somme en type entier ('int'). Étape 4 : Saisie sécurisée au clavier annotée par la flèche de flot de données // (clavier) >> saisir >> n. Étape 5 : Initialisation explicite de 'somme = 0' avant d'entrer dans la boucle pour purger les octets résiduels en RAM. Étape 6 : Construction d'une boucle 'for' allant de i = 1 jusqu'à n avec incrémentation de 1 à chaque tour et indentation de style Allman. Étape 7 : Accumulation de la valeur du compteur i dans la variable 'somme' (somme = somme + i). Étape 8 : Affichage du résultat final sur la sortie standard.

```
/*
  Fichier   : SommeEntiers.cpp
  Auteur    : Matéi
  Date      : 14/11/2026
  Objectif  : Calculer la somme des entiers de 1 a N
*/

#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES

    // VARIABLES
    int n;     // Limite saisie par l'utilisateur
    int i;     // Compteur de boucle
    int somme; // Accumulateur de calcul

    // TRAITEMENTS
    cout << "Veuillez saisir un nombre entier positif N : ";
    // (clavier) >> saisir >> n
    cin >> n;

    // Initialisation stricte pour purger la mémoire RAM de toute valeur poubelle
    somme = 0;

    // Boucle itérative bornée
    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    cout << "La somme des entiers de 1 a " << n << " est egale a : " << somme << endl;

    return 0;
}
```

Résultat : Pour une saisie utilisateur N = 5, le déroulement mémoire est le suivant : - Départ : somme = 0 - i = 1 → somme = 0 + 1 = 1 - i = 2 → somme = 1 + 2 = 3 - i = 3 → somme = 3 + 3 = 6 - i = 4 → somme = 6 + 4 = 10 - i = 5 → somme = 10 + 5 = 15 Le programme affiche à l'écran : 'La somme des entiers de 1 a 5 est egale a : 15'. L'algorithme est déterministe et conforme aux exigences académiques de l'IUT.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale de critère de choix entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est choisie lorsque le nombre précis d'itérations est connu à l'avance (structure bornée et déterministe). La boucle TANT QUE est choisie lorsque le nombre de répétitions dépend d'une condition dynamique réévaluée à chaque tour (structure non bornée).

Q : Que valent les variables A et B après l'exécution exacte de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : Après 'A ← A + B', A prend la valeur 5 + 3 = 8. Après 'B ← A - B', B prend la valeur 8 - 3 = 5. À la fin de la séquence, A vaut 8 et B vaut 5 (cette séquence permet d'échanger les valeurs de deux variables sans variable temporaire).

Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) entraîne-t-elle systématiquement la perte de points en partiel ?

R : Parce qu'une variable non initialisée contient une 'valeur poubelle' (données résiduelles non nettoyées en RAM). Si on calcule 'somme ← somme + i' sans initialiser 'somme' à 0, l'accumulateur ajoute i à un nombre aléatoire, ce qui produit un résultat totalement faux.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765